

Category: Developer Productivity / API Cost Analytics**Target users:** API-key agent users.

1. App idea and user problem

TokenTrail is an Android API-cost companion for students and independent developers who use API-key coding agents: **Codex with OpenAI**, **ZCode with general Z.ai API access** and a DeepSeek-side tool to be selected. A small pet floats **within TokenTrail's screens**; tapping it opens an expandable question panel. This **advice agent** explains cost changes or suggests a coding agent using logged usage, budgets and task ratings. TokenTrail **does not run, route or control** the coding agents.

The scope is **API usage priced at public API rates**; ChatGPT or GLM Coding Plan subscriptions and credits are excluded. A model response reports tokens for one call, not the agent's full history. TokenTrail imports available usage logs and prices a call only when a saved official-rate snapshot covers its date; otherwise historical cost is unavailable. An API key alone cannot recover another Agent's past per-call usage. Every monetary value is labelled an *estimate*, never an invoiced charge.

2. Related products and opportunity

The review compares first-party API billing views with existing cross-model monitoring tools on data scope, cost provenance and decision support.

Product	Relevant capability	Gap addressed by TokenTrail
Codex	Can run with an OpenAI API key, billed through the Platform account.	TokenTrail estimates only logged Codex API calls using dated OpenAI prices.
ZCode	Can use the general Z.ai API endpoint with an API key, separate from Coding Plan.	TokenTrail prices logged general-API calls, excluding plan usage.
dsh-context	DeepSeek Harness plugin showing local sessions, token use, cache, activity and estimated costs.	TokenTrail spans API providers; session detail appears only when usable logs are imported.
LiteLLM	Open-source gateway with routing, spend tracking and budgets for proxied calls.	TokenTrail does not proxy traffic; it tracks agent-associated API costs from permitted records.
dsh-pet	Interactive floating pet driven by DSH session events, with touch and feeding actions.	TokenTrail adds cross-provider, evidence-linked advice through an Android in-app pet.

3. Proposed contribution

Unified view. TokenTrail unifies authorised API-call logs from Codex, ZCode and a DeepSeek-side tool, de-duplicates records and shows their source and coverage; session detail appears only where logs support it. **Reproducible costs.** Java applies dated official rates by provider, model, token class and service tier, retaining the price version behind each estimate when rates change. **Decision support.** Activity, budgets and ratings of comparable tasks help users understand spending and choose an agent. The in-app floating pet is the conversational interface: read-only tools supply calculated evidence, and the model explains advice with sample size and missing-data limits. It neither controls coding agents nor encourages extra token use. Values remain *Estimated*, *Partial estimate* or *Unavailable*.

Scope rule: API-cost records, comparisons and the in-app advice pet are core. The pet can recommend but cannot execute coding agents or enforce a hard cap. Subscription quotas, public community, live cross-Agent status and system-wide overlays are outside the first release.

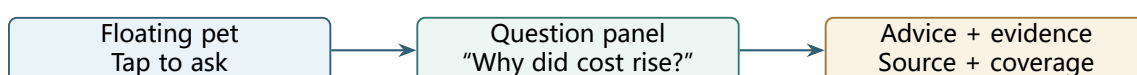
4. Planned functionality and the four data aspects

Aspect	Planned implementation	User value / evidence
Data input	Coding-agent profile, key label, budget and task rating; authorised usage logs; user questions to the pet.	Codex, ZCode and a DeepSeek-side tool; explicit source and coverage labels.
Data processing	Java prices dated usage and compares tasks; a pet agent requests read-only summaries and drafts advice.	Advice cites data and exposes missing fields; no invented live activity.
Data storage	Firebase Authentication; UID-restricted Firestore; Room for usage, price versions, pet preferences and advice history.	Historic estimates retain the price version used at calculation time.
Data output	Java/XML cards, heatmap, sessions, budget alerts and an in-app floating pet that opens an advice panel.	A mobile view of spending and work patterns, with unavailable states for missing logs.

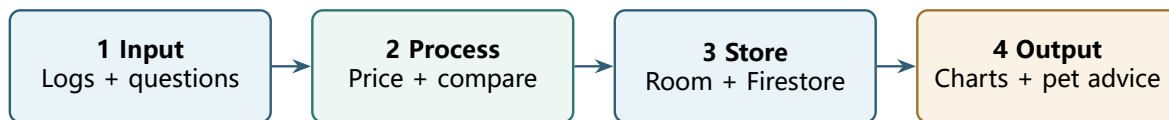
5. Core features and implementation approach

- **Account and agent profiles.** Registration, login, logout and password reset. Users add Codex, ZCode and a DeepSeek-side tool as tracked profiles, naming the API provider and a non-secret key label. TokenTrail does not request an agent password or dispatch tasks.
- **API usage and cost.** Import user-authorised per-call usage logs or exports; no Agent is required to send traffic through TokenTrail. Record call time, exact model, Agent, provider, input, cached-input and output tokens where available. De-duplicate calls, apply a matching saved price version when available and show the native-currency estimate; CNY display uses a dated exchange-rate snapshot.
- **Session insights and efficiency.** When logs permit, group calls by session and show title, time, turns, tokens, cache hit, tool calls and active duration. Session cost is the sum of eligible call estimates. If only aggregate tokens are available, show provider-level estimates without inventing sessions. Recommendations compare similar rated tasks and disclose sample size.
- **Budget and action.** Compare estimated spend with a user budget, showing coverage and uncertainty. Forecast only from comparable usage history; alerts are advisory, not a hard cap on external agents. TokenTrail handles no payment.
- **Pet advice agent.** A small animated pet floats within the app; tapping it opens an expandable question panel. It can also signal a rule-based budget warning. A Java service calls one selected external GPT, GLM or DeepSeek model API for the first release. Through function calling it may request *getUsageSummary*, *getBudgetStatus* and *compareAgents*; Java executes and validates these read-only tools. Replies state the evidence and missing data. No logs means no cost-based recommendation. The pet's own model tokens and estimated cost are tracked separately from coding-agent usage.
- **API price updates.** Store official-rate snapshots with source URL, currency, model, token class, service tier and recorded effective or observation time. Check pricing pages regularly and record changes. New rates apply prospectively; old estimates retain their original version. If an old call has no reliable contemporaneous rate, label a current-price comparison as uncertain rather than backdate it.

Pet interaction sketch. The panel opens from the in-app pet and links each answer to calculated evidence.



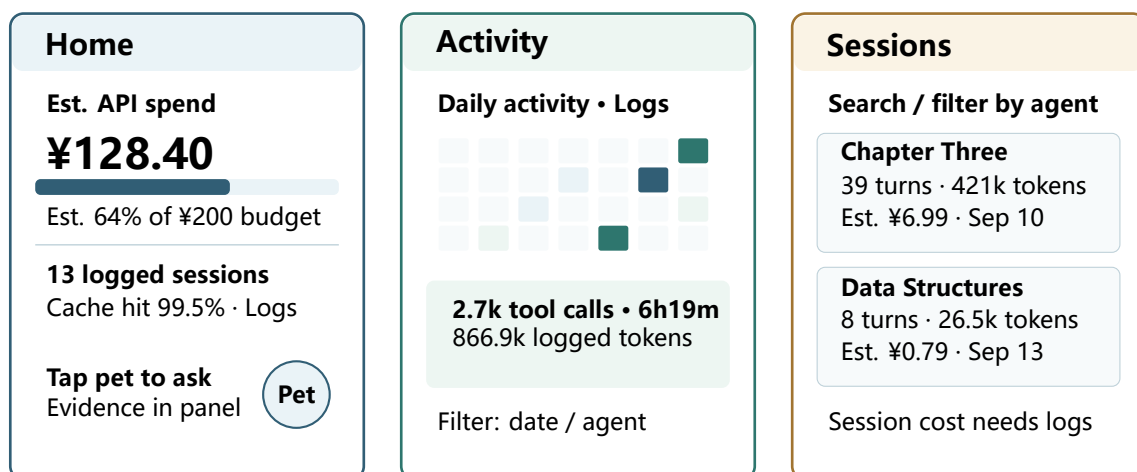
6. Data flow and limitations



Model responses report usage only for calls observed by the caller, so imported Agent logs are the primary input. Java matches token classes to dated prices and computes budgets; the external model receives selected summaries rather than API secrets or prompt text. A server-side function-call loop lets the pet query read-only tools, then return a short recommendation with source and coverage. The pet cannot observe a live coding session without a separate event feed.

7. Proposed Android interface

The main navigation has **Home, Activity, Sessions, Compare and Budget**. Inspired by dsh-context and dsh-pet, these mobile sketches show summary metrics, a heatmap and session cards. The pet floats within the app and opens a question panel on tap; that panel can expand to show evidence. Log-dependent sections show an empty state when records are unavailable.



Interaction and accessibility. Each screen shows loading, empty and error states. Figures carry Agent, provider, estimate status, usage coverage, price version and refresh time; the heatmap has a numeric summary. A context ring is not API spend. The team checks touch targets, text scaling and screen rotation.

8. Technology and responsible reuse

Layer	Planned tools and rationale
Android interface	Java/XML, Material Components and Navigation; ViewModel/LiveData for state; RecyclerView and MPAndroidChart for trends; an in-app floating pet view and expandable question panel.
Network and work	Retrofit/OkHttp for log import and a Java advice service. The service runs one model API and a bounded function-call loop over read-only usage, budget and comparison tools. WorkManager schedules price checks.
Persistence and security	Room plus UID-restricted Firestore; the Java service verifies Firebase ID tokens and holds its model key server-side. Imported metadata is protected; coding-agent passwords and keys are not required.
Open-source reuse	Room with a View guides storage structure; MPAndroidChart may supply charts. dsh-pet inspires interaction, but original or properly licensed art will be used. Reused files and licences will be recorded.

9. Work plan through the final submission

The three-person team will follow the approximate course checkpoints: Alpha (weeks 8–9), Beta (weeks 11–12) and Final (weeks 15–16). The team will commit to GitHub throughout the project and adjust scope based on tested progress.

Course weeks	Work	Checkpoint / acceptance
1–3	Confirm API-key Agent use cases and UI, set up Firebase Auth and Firestore rules, document usage-log access and official pricing dimensions.	Neither test account can access the other' s records; token-source limits are documented.
4–7	Implement Agent profiles, usage-log import, Room/Firestore records, versioned pricing, CNY display and dashboard.	Sample calls from each provider are priced with the correct dated rate and de-duplicated.
8–9: Alpha	Deliver a working login → add Agent → import token log → estimated cost → analytics path; distinguish real logs from sample data.	Source, description, GitHub link and MP4 demonstration per submission guidance.
10–12: Beta	Add budgets, sessions and task ratings; build the in-app floating pet and tap-to-open panel; connect one model to three read-only tools.	Pet answers a usage question with evidence and admits missing data; test price changes and errors.
13–15: Final	Evaluate advice accuracy and API cost, improve accessibility, verify account isolation and document estimation limits.	End-to-end demo and report/video; no pet answer invents usage or controls coding agents.

10. Team work and evidence of completion

The team split is **Zhang Li (24107757)**: UI/UX design, Java/XML screens, charts and accessibility; **Wang Tingdong (24107759)**: authentication, usage-log import, Room/Firestore, versioned pricing and budget computation; **Liu Zongrun (24107745)**: in-app floating pet, question panel, external-model service, read-only tools and advice evaluation. All three review integrations and contribute GitHub commits.

The final demo should show: account isolation; tracked Codex, ZCode and DeepSeek-side profiles; three provider estimates from dated token logs; a heatmap and budget warning; then a pet question such as “Why did cost rise?” answered with a queried summary, evidence and its own estimated API cost.

Validation will test token accounting, price-version boundaries, missing usage, de-duplication and read-only tool limits; the Firebase Emulator verifies access. Pet replies are checked against tool data. Published rates can differ from paid amounts; all figures remain estimates.

11. Sources and project references

API scope	OpenAI pricing; DeepSeek pricing; Z.ai pricing; Codex billing modes; ZCode endpoints.
Agent	Function calling; Java API client; Firebase ID-token verification.
Related	dsh-context dashboard; dsh-pet; LiteLLM; Room with a View; MPAndroidChart.
Security	Firestore Security Rules conditions.